

NUMERICAL SIMULATIONS OF TIME-DEPENDENT PROBLEMS

Gabriel TURINICI, 2026

DO NOT DISTRIBUTE

Chapter 1

Motivations and Examples: Epidemiology, Finance, Deep Learning

The object of this book is the presentation of numerical algorithms related to situations where the **time** plays an important role.

Natural examples are finding the solutions to time-dependent problems, be them expressed as deterministic ordinary differential equations (denote ODE from now on) or Stochastic Differential Equations (denoted SDE from now on). We will further see that finding derivatives in a computational graph also share some important features with the time-dependent problems, namely the notion of causality and in particular we will apply these latter techniques to the control of evolution equations and also to techniques in statistical (deep) learning.

We begin with some examples of applications.

1.1 ODE : Ordinary Differential Equations

An important model in epidemiological modeling is the SIR model [4] proposed in 1927 by W.O. Kermack A.G. McKendrick and that was used and improved in a myriad of contributions since; the initial S stands for the number of “susceptible” individuals, I for the group of “infected” individuals, and R for the group of those that “recovered”, see Figure 1.1 for a graphical representation. After a derivation which will be presented later in Section 2.7, together with the main underlying hypothesis, we obtain the system of equations, called the



Figure 1.1: Schematic representation of the SIR model in equation (1.1).

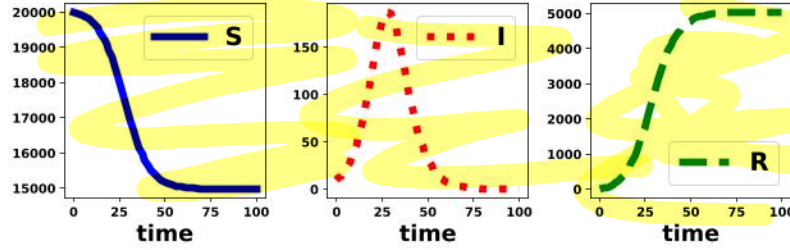


Figure 1.2: Typical evolution of the system in Equation (1.1); data taken from [7].

SIR model

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} \\ \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I \\ \frac{dR}{dt} = \gamma I. \end{cases} \quad (1.1)$$

We assume $S(0) = S_0 \neq 0$, $I(0) = I_0 > 0$, $R(0) = R_0 \geq 0$, $S_0 + I_0 + R_0 = N$, N is the total population. Here, β , γ are parameters of the model. A typical evolution is given in Figure 1.2.

In reality, the model needs to be adapted, as real data is not always compatible with simple models, see Figure 1.3. Therefore, we move beyond the domain of models with analytical solutions and must find accurate numerical approximations of their solutions. The study of the numerical procedures to find such solutions, called “numerical schemes” is the object of chapter 2. We will study important notions such as the consistency, stability and convergence of the schemes. Several types of schemes will be presented including Runge-Kutta, multi-step schemes and adaptive schemes.

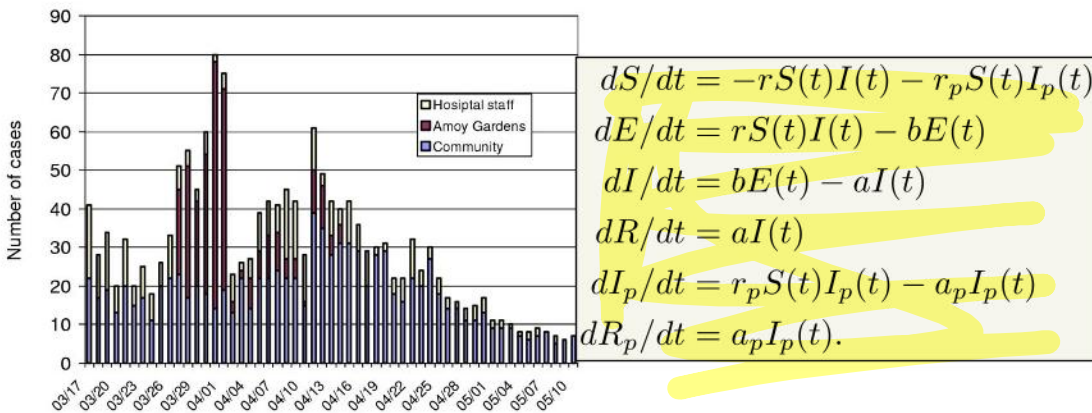


Figure 1.3: Actual evolution of the number of infected individuals; image taken from [7]. To accurately reproduce real data, it is necessary to use a model like the one on the right. But this model does not have analytical or semi-analytical solution any more.

1.2 SDE : Stochastic Differential Equations

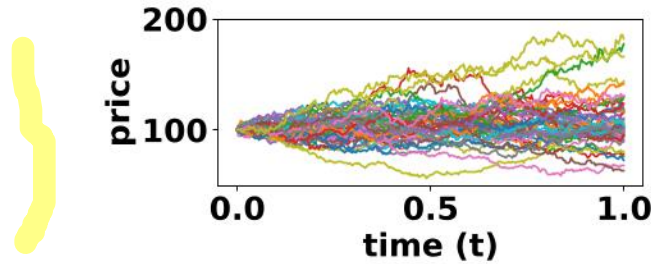


Figure 1.4: Solution scenarios for (1.2). Abscissas represent time and ordinates represent prices. If the random argument is denoted ω each scenario plotted here represents a function $t \mapsto S_t(\omega)$ for some given ω .

In financial applications (calculations for derivative products in different scenarios) or in physics (path integrals, etc.), there is a need to handle quantities that evolve over time and also contain an element of uncertainty. For instance, the yield $\frac{S_{t+\Delta t} - S_t}{S_t}$ of a financial asset whose price at time t is denoted S_t contains a predictable part and also a random part; this can be modeled, like in the log-normal (sometimes called Black-Scholes) model [3, 8], as a normal variable $\mathcal{N}(\mu\Delta t, \sigma^2\Delta t)$. We do not discuss the justification or the validity of this model here; for real-life applications, the model must be carefully validated !! In particular this is not an encouragement to use any such model in practical situations, we only present a “cultural” vision of the mathematical finance models. The following SDE is obtained (see [10] for details):

$$dS_t = \mu S_t dt + \sigma S_t dW_t. \quad (1.2)$$

An illustration of a solution, represented in the “Wiener space” as a set of scenarios, for (1.2) is provided in Figure 1.4.

Recall that derivative products are financial instruments whose value depends (according to a pre-established contract) on an underlying asset. Example: the owner of a European call option on S_t with “strike” K and “maturity” T will receive a final payment of $(S_T - K)_+$ at time T . Note that this is random variable depending on S_T . In particular a profit is made only if $S_T > K$ so buying such a product is a bet that at the final time T the price S_T of the underlying S will exceed K . The calculation of the value of such a derivative contract **before** expiration T is unknown. Models, which are but imperfect representations of the reality, need to be chosen, and quantities such as:

$$\mathbb{E}^{\mathbb{Q}}[e^{-r(T-t)}(S_T - K)_+ | (S_u)_{u \leq t}], \quad (1.3)$$

need to be calculated. Since S_t follows an SDE the goal of the SDE chapter 4 is to calculate solutions of SDEs, study the accuracy of numerical calculations, determine if precise scenario calculations are desired (strong convergence) or only averages (weak convergence), etc...

1.3 Automatic differentiation: computational graphs, control

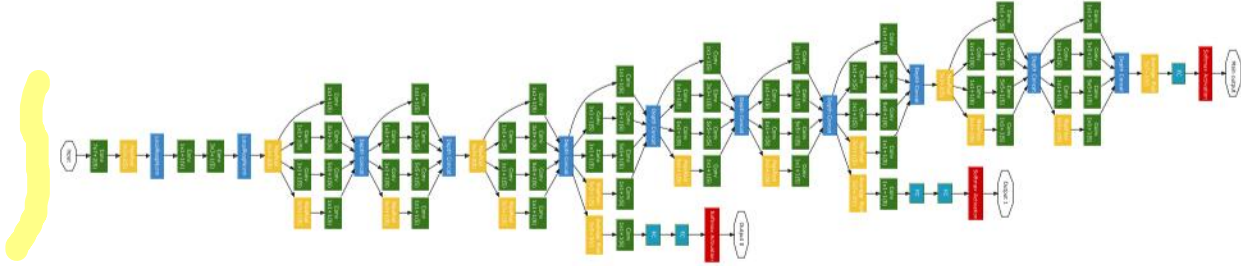


Figure 1.5: "Inception" network architecture from [9]. Each cell represents a multi-variable calculation such as matrix-vector multiplication followed by a non-linear operation like taking the positive part on each component. Some cell structures are repeated, for instance there are 9 9 identical inception blocks and some initial and final layers.

We now turn to less obvious applications of temporal (in the sense of causal) dynamics. Often the goal of a scientific approach is to influence the evolution of a system by acting on various parameters called "controls". To do so, many algorithms use the information given by the sensitivity of the outcome with respect to the "controls" which in mathematical terms is called the derivative. So we want to compute the derivative of a result (obtained by solving numerically some evolution equation) with respect to some input parameters; an example can be constructed from section 1.1 if we want to know how $S(\infty)$ of (1.1) depends on β .

This approach can be much further generalized. In its very abstract form, we see the result as obtained using sequential calculations presented in some order. This is formalized as a computational graph; here the derivative of the output of the graph computation with respect to the inputs has to be calculated. The technique is called "automatic differentiation" or "backpropagation"; in control theory, this gives rise to "adjoint states".

To be less abstract, consider the following simple example adapted from [6], see also [2, chap 6.5]: the graph has inputs x, y, z , output $f = f(x, y, z)$ and the computation is expressed as $f = 5 \cdot (x + y \cdot z)$. To calculate $\partial_x f, \partial_y f, \partial_z f$, write it as a computational graph (direct or "forward" calculation):

$$u = y \times z \tag{1.4}$$

$$v = x + u \tag{1.5}$$

$$f = 5 \times v \tag{1.6}$$

Assume some values for the inputs are given, for instance $x = 1, y = 2, z = 3$; here are the relations obtained by elementary derivation of each calculation (adjoint or "backward" calculation). Note that, for instance is easier to compute $\partial_v f$ than $\partial_z f$ so the computation is done "backward", that is we obtain the derivative with respect to most recent outputs

first :

$$\partial_v f = 5 \quad (1.7)$$

$$\partial_x f = \partial_v f \times \partial_x v = \partial_v f = 5 \quad (1.8)$$

$$\partial_u f = \partial_v f \times \partial_u v = \partial_v f = 5 \quad (1.9)$$

$$\partial_y f = \partial_u f \times \partial_y u = 5z = 15 \quad (1.10)$$

$$\partial_z f = \partial_u f \times \partial_z u = 5y = 10. \quad (1.11)$$

Of course, the previous example is simple and straightforward. In practice operational examples comprise millions of operations and no human can write down by hand the full set of derivatives. Some automatic procedure is needed. This is not a miracle but a patient and rigorous application of the chain rule of derivation and will be presented in chapter [3](#). We will also study the relationship between the derivative and the computational graph and observe the emergence of an auxiliary variable called the “adjoint state”. It is crucial for the formalization of the calculation and allows the treatment of complex situations (cf. Figure [1.5](#) for the “Inception” network [\[9\]](#)).

Chapter 2

Ordinary Differential Equations (ODE)

We are concerned in this chapter with ODEs. Let I be an open interval included in $\mathbb{R}_+$. The following ordinary differential equation will be considered :

$$\frac{dX}{dt} = f(t, X(t)), \quad X(t_0) = X_0, \quad (2.1)$$

or, in integral form, as :

$$X(t) = X(t_0) + \int_{t_0}^t f(s, X(s)) ds. \quad (2.2)$$

2.1 Existence and Uniqueness of the Solution

Note that (2.1), called a “Cauchy problem” is indeed an equation because the unknown $X(t)$ appears in both sides of the expression. As for any equation, the existence of a solution is not automatic. To clarify the existence and uniqueness of the solution to the previous ODE, we use the following two theorems (please consult your favorite differential equations books for reference):

Theorem 2.1 (Local Lipschitz variant of Cauchy-Lipschitz). *Let $f : I \times \mathbb{R} \rightarrow \mathbb{R}$ be a locally Lipschitz continuous function at $X_0 \in \mathbb{R}$, $t_0 \in I$. In other words, there exist two balls $B_x(X_0, R_x)$, $B_t(t_0, R_t)$ and a constant $L > 0$ such that $\forall t \in B_t(t_0, R_t)$, $\forall X_1, X_2 \in B_x(X_0, R_x)$:*

$$|f(t, X_1) - f(t, X_2)| \leq L|X_1 - X_2|.$$

Then there exists $\varepsilon > 0$ such that the Cauchy problem (2.1) has a unique local solution: $X(t) : (t_0 - \varepsilon, t_0 + \varepsilon) \subset I \rightarrow \mathbb{R}$. Moreover, $X(\cdot)$ is a C^1 function.

Theorem 2.2 (Global Lipschitz variant of Cauchy-Lipschitz). *Under the same assumptions as in Theorem (2.1), if L is the same for all R_x (radius of the ball) and initial condition X_0 , then a global solution exists and is unique.*

Remark 2.3. Global existence also holds if we can find a continuous function $\alpha : \mathbb{R} \rightarrow \mathbb{R}_+$ such that

$$|f(t, X_1) - f(t, X_2)| \leq \alpha(t)|X_1 - X_2|.$$

This allows L to depend on time.

We will now consider some examples.

Example 2.4 (linear function). Let $f(t, X) = rX$ with $r \in \mathbb{R}$ constant. Then $|f(t, X_1) - f(t, X_2)| = |r| \cdot |X_1 - X_2|$, so we obtain global existence with $L = |r|$.

Example 2.5 (non-linear function that blows up). Let $f(t, X) = \frac{5}{X-3}$. An immediate calculation gives $|f(t, X_1) - f(t, X_2)| = \frac{5}{|(X_1-3)(X_2-3)|} \cdot |X_1 - X_2|$, so we obtain local existence for $L = \sup_{X_1, X_2 \in \mathcal{V}} \frac{5}{|(X_1-3)(X_2-3)|}$ in an open neighborhood $\mathcal{V}$ of any point $X_0 \neq 3$ (such that $3 \notin \overline{\mathcal{V}}$). However, as $\frac{5}{|(X_1-3)(X_2-3)|}$ is not bounded around $X_0 = 3$, the global existence theorem is not applicable at $X_0 = 3$.

2.2 Numerical Methods

A "numerical scheme" is a procedure to solve the ODE. It is formulated **independent** of the function f that actually defines the ODE (2.1). We do not look for numerical schemes that only work for particular ODEs but for schemes that work for a large class of functions f . So, unless stated otherwise, the regularity of f is not the focus of this chapter (see nevertheless the "adaptive time steps" and "stiff systems" sections 2.8.1 and 2.8.4) so we can suppose f at least C^1 and certainly at least locally Lipschitz. Then the solution to the Cauchy problem exists and it is unique (cf. Theorem 2.1). When there is no analytic solution, the solution will only be known numerically, we will approximate it using different methods. The approximation is done, for example, on $[0, T]$ with N points.

2.2.1 Important notations

We introduce some important notations used throughout this chapter :

- the equation to solve is (2.1);
- $h := T/N$ is called the "time step"; we denote $t_n := n \cdot h, \forall n \leq N$;
- $X_n := X(t_n)$ is the exact solution; in general we do not have a analytic formula so X_n will remain unknown;
- U_n will be an approximation of X_n ; this is the main object that we search
- $f_n := f(t_n, U_n)$.

2.2.2 One step methods

So the main question is how to calculate the U_n ? For example, starting from the following formula:

$$X(t_{n+1}) = X(t_n) + \int_{t_n}^{t_{n+1}} f(s, X(s)) ds. \quad (2.3)$$

one could imagine a recurrence that is called a **one-step method** given by :

$$U_{n+1} = U_n + h\phi(t_n, U_n, f_n, h). \quad (2.4)$$

Each function ϕ gives another numerical method. Note that ϕ can also depend on U_{n+1} or f_{n+1} ; in this case, we speak of implicit methods.

2.2.3 Definition of four classic numerical schemes

We describe below four important numerical schemes. To illustrate their use we take two particular examples of function f namely $f_1(t, X) = rX$ and $f_2(t, X) = rX^2$. We recall that for f_1 , the solution $X(t)$ of $\dot{X}(t) = f_1(t, X(t))$ is $X(t) = e^{rt}X_0$, while for f_2 , the solution $Y(t)$ of $\dot{Y}(t) = f_2(t, Y(t))$ is $Y(t) = \frac{Y_0}{1-rtY_0}$.

- **Explicit Euler (denoted EE from now on):**

$$\begin{cases} U_{n+1} &= U_n + hf(t_n, U_n) = U_n + hf_n \\ U(0) &= X(0) \end{cases} \quad (2.5)$$

Here, $\phi = f_n$. Examples: for f_1 : $U_{n+1} = U_n + hrU_n = (1+rh)U_n$; for f_2 : $U_{n+1} = U_n + hrU_n^2 = (1+rhU_n)U_n$.

- **Implicit Euler (denoted IE from now on):**

$$\begin{cases} U_{n+1} &= U_n + hf_{n+1} \\ U(0) &= X(0) \end{cases} \quad (2.6)$$

Here, $\phi = f_{n+1}$. Examples: for f_1 : $U_{n+1} = U_n + hrU_{n+1}$ so $U_{n+1} = \frac{U_n}{1-rh}$; for f_2 : $U_{n+1} = U_n + hrU_{n+1}^2$ so U_{n+1} is a solution of $rhU_{n+1}^2 - U_{n+1} + U_n = 0$.

When f is Lipschitz, for sufficiently small h , the value U_{n+1} , a solution of the implicit IE scheme definition equation, is unique, see exercise 2.7 page 27.

- **Crank-Nicolson (denoted CN from now on, implicit):**

$$\begin{cases} U_{n+1} &= U_n + h \left[\frac{f_n + f_{n+1}}{2} \right] \\ U(0) &= X(0) \end{cases} \quad (2.7)$$

Examples: for f_1 : $U_{n+1} = U_n + hr \frac{U_n + U_{n+1}}{2}$ so $U_{n+1} = \frac{1+\frac{rh}{2}}{1-\frac{rh}{2}} U_n$; for f_2 : $U_{n+1} = U_n + hr \frac{U_n^2 + U_{n+1}^2}{2}$ so U_{n+1} is a solution of $\frac{rh}{2} U_{n+1}^2 - U_{n+1} + (1 + \frac{rh}{2} U_n) U_n = 0$.

- **Heun (denoted H from now on, explicit):**

$$\begin{cases} U_{n+1} &= U_n + \frac{h}{2} [f_n + f(t_{n+1}, U_n + hf_n)] \\ U(0) &= X(0) \end{cases} \quad (2.8)$$

Examples: for f_1 : $U_{n+1} = U_n + \frac{h}{2} [rU_n + r(U_n + hrU_n)]$; for f_2 : $U_{n+1} = U_n + \frac{h}{2} [rU_n^2 + r(U_n + hrU_n)^2]$.

Intuition 2.6. The relation (2.3) indicates that we need to find a way to approximately calculate the integral of $f(t, X(t))$ between t_n and $t_{n+1} = t_n + h$. The EE scheme takes an approximation using the method of rectangles by using the value at t_n , the IE scheme uses the value at $t_n + h$, and the CN scheme takes the average of the two, i.e., it uses the trapezoidal rule. As for the Heun scheme, it uses an approximation of X_{n+1} that it reintroduces into a CN-type scheme but with the idea of keeping it explicit.

2.3 Error, Consistency, and Order

2.3.1 Error

When introducing the exact solution into the formula (2.4) for one-step methods, we obtain "truncation errors" $\tau_{n+1}(h)$:

$$\underbrace{X(t_{n+1}) = X(t_n) + h\phi(t_n, X_n, f(t_n, X_n), h)}_{\text{true for the numerical scheme, i.e., } U_n \text{ instead of } X_n, \text{ etc.}} + h\tau_{n+1}(h).$$

or

$$\begin{aligned} \tau_{n+1}(h) &:= \frac{X(t_{n+1}) - X(t_n) - h\phi(t_n, X_n, f(t_n, X_n), h)}{h} \\ &= \frac{X(t_{n+1}) - X(t_n)}{h} - \phi(t_n, X_n, f, h). \end{aligned} \quad (2.9)$$

Definition 2.7. The remainder that appears when the true solution is placed into the relation defining the numerical scheme (similar in form to the initial equation) is called the truncation error. For one-step methods (2.4), it is $\tau_{n+1}(h)$ defined in (2.9), which is called the local truncation error at step $n + 1$. The global truncation error is defined by the relation: $\tau(h) = \max_{n=1, \dots, N} |\tau_n(h)|$.

Remark 2.8. The truncation error here is the same as the error (divided by h) between X_{n+1} and the U_{n+1}^* obtained starting from $U_n = X_n$.

Example 2.9. Explicit Euler: Using the Taylor series formula to the 2nd order:

$$X(t+h) = X(t) + h\dot{X}(t) + \frac{1}{2}h^2\ddot{X}(\xi), \quad \xi \in [t, t+h].$$

For $t = t_n$ and $t_n + h = t_{n+1}$, we get the existence of some $\xi_n \in [t_n, t_{n+1}]$ such that : $\tau_{n+1}(h) = \frac{1}{2}h\ddot{X}(\xi_n)$.

Implicit Euler: : similar computations can be done but also check the technique (2.14) and the exercise (2.7) page (27).

2.3.2 Consistency and Order

Definition 2.10. A scheme is said to be consistent if:

$$\lim_{h \rightarrow 0} \tau(h) = 0, \quad (2.10)$$

i.e., for small h , the exact solution satisfies the scheme. A scheme is called of order " p " if: $\tau(h) = O(h^p)$ for $h \rightarrow 0$.

So we just proved that EE and EI consistent and of first order.

2.4 Stability and Convergence

2.4.1 Zero-Stability

When doing numerics on a computer various round-off errors influence the exactitude of the result. No computation is exempted so in practice we do not have access to the exact values as defined in the numerical schemes. To study stability with respect to such round-off errors we treat them as perturbations and check if Z_n^h defined by:

$$\begin{aligned} Z_{n+1}^{(h)} &= Z_n^{(h)} + h[\phi(t_n, Z_n^{(h)}, f(t_n, Z_n^{(h)}), h) + \delta_{n+1}] \\ Z_0^{(h)} &= \delta_0 + X_0, \end{aligned} \quad (2.11)$$

is close to U_{n+1} .

Advanced Topics 2.11. Numerical inaccuracies are very small during a simple addition operation but mostly occur in the computation, often complex, of the function ϕ ; that's why the perturbations δ_n are placed where indicated in the formula (2.11). For example, if $f(t, X) = X^2 - 1$ needs to be calculated at $t = 0, X = \sqrt{2}$, the value $\sqrt{2}^2 - 1 = 1$ is often affected by errors. Below is an example implemented in Python :

```
In [2]: numpy.sqrt(2)**2 -1
Out[2]: 1.0000000000000004
```

So an error of order 10^{-16} occurs even for this simple case.

A legitimate question is whether the errors accumulate during computations of the numerical schemes. Of course, when the input has already some error the numerical scheme will not be able to correct it. But a good property would be to ensure the order of the error remains roughly the same. This motivates the following definition.

Definition 2.12. The scheme given by ϕ is called zero-stable if there exists h_0 and a constant C (independent of ε) such that if $h \leq h_0$ and $|\delta_n| < \varepsilon$ ($\forall n$): then

$$|Z_{n+1}^{(h)} - U_{n+1}| \leq C\varepsilon, \quad \forall n \geq 0. \quad (2.12)$$

We give now sufficient conditions for zero-stability.

Theorem 2.13. Assuming f and ϕ are Lipschitz with respect to their second variable, meaning that there exist $\Lambda > 0, h_0 > 0$ such that $\forall h < h_0$ such that :

$$|\phi(t, X, f(t, X), h) - \phi(t, Y, f(t, Y), h)| < \Lambda|X - Y|, \quad \forall X, Y.$$

Then the numerical scheme given by ϕ is zero-stable.

Proof. Let's denote: $W_n = Z_n^{(h)} - U_n$. Then

$$W_{n+1} = Z_{n+1}^{(h)} - U_{n+1} = h[\phi(t_n, Z_n^{(h)}, f, h) - \phi(t_n, U_n, f, h)] + h\delta_{n+1}$$

so $|W_{n+1}| \leq |W_n| + h\Lambda|W_n| + h|\delta_{n+1}|$ thus by summing these inequalities and simplifying terms:

$$|W_{n+1}| \leq |W_0| + h\Lambda \sum_{s=0}^n |W_s| + \sum_{s=1}^{n+1} h|\delta_s|.$$

This allows us to conclude using the discrete Gronwall's lemma (see exercise 2.3 page 26):

$$|W_{n+1}| \leq (|W_0| + hn\epsilon) \cdot \exp(h\Lambda n) \leq (1 + T)\epsilon \exp(\Lambda T).$$

□

Important technique 2.14. Question: which methods among EE, IE, CN, H satisfy the assumptions of theorem 2.13?

- EE: $\phi = f_n$, is Lipschitz when f is.
- H: similar techniques
- For general implicit methods, see exercise 2.7 page 27.
- Intuition for IE: by definition, ϕ has the property: $\phi(t_n, U_n, f_n, h) = f(t_{n+1}, U_{n+1})$ (assuming the existence of a unique solution). Then for two initial points U_n, V_n , we need to bound $f(t_{n+1}, U_{n+1}) - f(t_{n+1}, V_{n+1})$ by some constant times $|U_n - V_n|$. The difficulty here is to obtain $|U_n - V_n|$ and not $|U_{n+1} - V_{n+1}|$. To this end we write: $|f(t_{n+1}, U_{n+1}) - f(t_{n+1}, V_{n+1})| \leq L|U_{n+1} - V_{n+1}|$ and $|U_{n+1} - V_{n+1}| \leq |U_n - V_n| + h|f(t_{n+1}, U_{n+1}) - f(t_{n+1}, V_{n+1})| \leq |U_n - V_n| + hL|U_{n+1} - V_{n+1}|$; thus, after moving $hL|U_{n+1} - V_{n+1}|$ to the left hand side we obtain $|U_{n+1} - V_{n+1}| \leq |U_n - V_n|/(1 - hL)$. From here we get the conclusion with constant $L/(1 - hL)$.

- CN: similar techniques cf. TD!!

2.4.2 Convergence

Now that we are reassured about round-off errors, we can use these techniques to the main question of whether we can be as close to the true solution as we want. This is the notation of convergence.

Definition 2.15. A scheme is said to be convergent at order p if, with the previous notations, $|U_n - X_n| = O(h^p)$. A scheme convergent at order 1 is simply called "convergent."

Theorem 2.16. Under the same assumptions as in Theorem 2.13, we have:

$$|U_n - X_n| \leq (|U_0 - X_0| + nh\tau(h)) \cdot \exp(\Lambda nh).$$

In particular, if for $p \geq 1$: $|U_0 - X_0| = O(h^p)$ and $\tau(h) = O(h^p)$, then $|U_n - X_n| = O(h^p)$ (the scheme converges at order p).

Proof. We follow the same steps as in the proof of Theorem 2.13, with $\delta_j = \tau_j(h)$ (using the discrete Gronwall's lemma). Here, the exact solution X_n plays the role of the perturbation $Z_n^{(h)}$. $\square$

Advanced Topics 2.17. *The previous theorem can be rewritten as stating that **consistency and stability imply convergence**. This is a principle often encountered.*

Corollary 2.18. *The schemes EE and IE converge at order 1. The schemes CN and H converge at order 2.*

Proof. I will provide detailed reasoning only for the Crank-Nicholson scheme; for the Heun scheme the result will be proved in a more general setting in the Proposition 2.30 and Example 2.31.

$$X_{n+1} = X_n + \frac{h}{2} \left[f(t_n, X_n) + f(t_{n+1}, X_{n+1}) \right] + h\tau_{n+1}(h) \quad (2.13)$$

so (for now, treating it as if it were explicit, see exercise 2.7 for details):

$$X_{n+1} = X_n + \frac{h}{2} \left[X'_n + X'_{n+1} \right] + h\tau_{n+1}(h). \quad (2.14)$$

Also, the Taylor series at order 2 for X' and at order 3 for X provide:

$$X'_{n+1} = X'_n + hX''_n + \frac{h^2}{2} X_n^{(3)}(\eta) \quad (2.15)$$

$$X_{n+1} = X_n + hX'_n + \frac{h^2}{2} X''_n + \frac{h^3}{6} X_n^{(3)}(\xi) \quad (2.16)$$

Replacing (2.15) and (2.16) into (2.14) gives:

$$h\tau_{n+1}(h) = \frac{h^3}{6} X_n^{(3)}(\xi) - \frac{h^3}{4} X_n^{(3)}(\eta) \quad (2.17)$$

which leads to $\tau_{n+1}(h) = O(h^2)$ (after some calculations to transfer the implicit version into an explicit one). $\square$

Advanced Topics 2.19. *So, we have several methods, each with its convergence order. Which one to choose then? A naive answer would be to pick the scheme with the highest order. However, as we saw with CN, for the order to be effective, we need to use higher derivatives of f (meaning f needs to be smooth), and on the other hand, the higher the order of the scheme, the more intermediate calculations of the f function are required (EE/IE have only one f calculation, while CN uses two), which can be costly. In practice, one rarely goes beyond order 4 or 5 (and sometimes sticks to order 1).*

2.4.3 Absolute Stability

Beyond convergence, stability is still an important property. We already inquired about a local notion of stability. Here we move to another notion and consider stability from the perspective of the solution over a long time $T = Nh$ (as $T \rightarrow \infty$), but for a fixed step size h (so $N \rightarrow \infty$). The analysis will employ the following prototypical situation; take $\lambda \in \mathbb{C}$, $t \geq 0$, and consider the test problem:

$$\dot{Y}(t) = \lambda Y(t) \quad (2.18)$$

$$Y(0) = 1 \quad (2.19)$$

with the solution $Y(t) = e^{\lambda t}$. For $\operatorname{Re}(\lambda) < 0$, we obtain $\lim_{t \rightarrow +\infty} Y(t) = 0$. So, any local perturbation in time is "erased" in the long run. This is a very desirable property for numerical schemes that have to fight rounding errors, etc. We want to preserve such a property.

Definition 2.20. A scheme is said to be absolutely stable if, for $f(t, x) = \lambda x$ and $\forall h, \lambda$, $U_n \rightarrow 0$. Otherwise, its region of absolute stability is:

$$\{h\lambda \in \mathbb{C} \mid U_n \rightarrow 0\}.$$

Advanced Topics 2.21. Choosing f to be linear is not so surprising because at first order $f(t, x) \simeq f(t, X_0) + \frac{\partial f}{\partial x}(t, X_0)(x - X_0)$ so, apart from a constant, we have a linear function.

Example 2.22 (Explicit Euler). For $f(t, x) = \lambda x$ we obtain $U_n = (1 + h\lambda)^n U_0$. We define the stability region by imposing the stability condition, see figure (2.1) for an illustration :

$$\mathcal{S}_{EE} = \{z \in \mathbb{C} : |1 + z| < 1\}. \quad (2.20)$$

Example 2.23 (Implicit Euler). Here direct computations show that $U_{n+1} = \frac{U_n}{1 - h\lambda} = \frac{U_0}{(1 - h\lambda)^{n+1}}$. To find the stability region, limit the time step h by imposing the stability condition, see figure (2.1) :

$$\mathcal{S}_{IE} = \{z \in \mathbb{C} : |1 - z| > 1\}. \quad (2.21)$$

Example 2.24 (Crank-Nicholson). Using the definition of the scheme we get:

$$U_{n+1} = \left(\frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}} \right)^{n+1} U_0. \quad (2.22)$$

The stability region is defined by imposing the stability condition:

$$\left\{ z = h\lambda \in \mathbb{C} : \left| \frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}} \right| < 1 \right\} = \{z \in \mathbb{C} : \operatorname{Re}(z) < 0\},$$

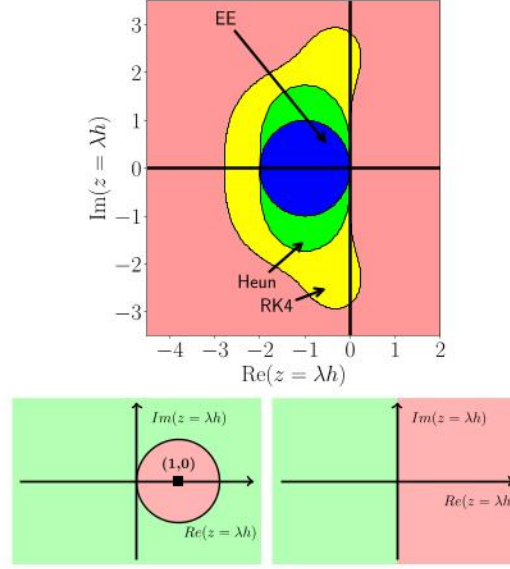


Figure 2.1: The stability domain of some numerical schemes. Region in pink is unstable. **Top image : explicit schemes**, stability region for Explicit Euler in blue, for Heun in green, for RK4 (2.34) in yellow; **Bottom images, implicit schemes** stability region in green, bottom left for Implicit Euler, bottom right for Crank-Nicholson.

thus we obtain, see figure 2.1 :

$$\mathcal{S}_{CN} = \{z \in \mathbb{C} : \operatorname{Re}(z) < 0\}. \quad (2.23)$$

Example 2.25 (Heun). The stability region is (after very similar calculations), see figure 2.1 :

$$\mathcal{S}_H = \left\{ z \in \mathbb{C} : \left| 1 + z + \frac{z^2}{2} \right| < 1 \right\}. \quad (2.24)$$

2.5 Higher-Order Methods: Runge-Kutta

These are methods that evaluate the function at intermediate steps:

$$U_{n+1} = U_n + hF(t_n, U_n; h, f). \quad (2.25)$$

with the function F of the scheme defined by

$$F(t_n, U_n; h, f) = \sum_{i=1}^s b_i K_i \quad (2.26)$$

$$K_i = f(t_n + c_i h, U_n + h \sum_{j=1}^s a_{ij} K_j), i = 1, 2, \dots, s, c_i \geq 0. \quad (2.27)$$

A method of this kind is called a Runge-Kutta (R-K) method. For a simpler presentation,

we introduce the **Butcher table of the scheme** $\begin{array}{c|c} c & A \\ \hline & b^T \end{array}$ or

$$\begin{array}{c|cccc} c_1 & a_{11} & a_{12} & \dots & a_{1s} \\ c_2 & a_{21} & a_{22} & \dots & a_{2s} \\ \dots & \dots & \dots & \dots & \dots \\ c_s & a_{s1} & a_{s2} & \dots & a_{ss} \\ \hline & b_1 & b_2 & \dots & b_s \end{array} \quad (2.28)$$

We will always assume $\sum_{j=1}^s a_{ij} = c_i$.

Definition 2.26. If A is strictly lower triangular, then the method is called explicit; if A is only lower triangular, then the method is semi-explicit. In all other cases, it is an implicit method.

Intuition 2.27. • When A is lower triangular with zero diagonal, the calculation of K_1 is done explicitly. Then this allows the explicit calculation of K_2 and so on. The scheme is thus explicit.

- When A is triangular with a non-zero diagonal, the method requires the sequential solution of s equations (not necessarily linear) to find the K_i , $i \leq s$.
- When A is full, the method requires the simultaneous solution of s equations (a system of equations) to find the K_i , $i \leq s$.

Remark 2.28. For implicit methods, one would also need to show the existence of a solution for each time steps. Assuming f is Lipschitz, this follows from Picard's fixed-point theorem by taking a recurrence as in Exercise 2.7.

Example 2.29 (4th-order R-K method). A remarkable Runge-Kutta example is the scheme:

$$U_{n+1} = U_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4) \quad (2.29)$$

$$K_1 = f_n = f(t_n, U_n), \quad (2.30)$$

$$K_2 = f(t_n + \frac{h}{2}, U_n + \frac{h}{2}K_1), \quad (2.31)$$

$$K_3 = f(t_n + \frac{h}{2}, U_n + \frac{h}{2}K_2), \quad (2.32)$$

$$K_4 = f(t_{n+1}, U_n + hK_3). \quad (2.33)$$

The associated Butcher table is

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array} . \quad (2.34)$$

This scheme is of order 4 (proof not given, accepted as true).

2.5.1 Construction of Second-Order Methods (Explicit)

We may inquire how can one construct new Runge-Kutta schemes. A partial answer is given below that concerns second order schemes.

Proposition 2.30. *Explicit methods with $s = 2$ that are second-order (in terms of convergence) satisfy $b_1 + b_2 = 1$ and $b_2 c_2 = 1/2$.*

Proof. Take $s = 2$. Since the method is explicit, the matrix A in the Butcher table is strictly lower triangular, i.e., $A = \begin{pmatrix} 0 & 0 \\ a_{21} & 0 \end{pmatrix}$. Let $a := a_{21}$, then $c_1 = 0 + 0 = 0$ and $c_2 = a + 0 = a$.

The corresponding Butcher table is:

0	0	0
a	a	0
	b_1	b_2

Then $U_{n+1} = U_n + hF$ with: $F = b_1 K_1 + b_2 K_2$, $K_1 = f(t_n, U_n) = f_n$, $K_2 = f(t_n + ah, U_n + haK_1) = f(t_n + ah, U_n + haf_n)$. So, $U_{n+1} = U_n + h(b_1 f_n + b_2 f(t_n + ah, U_n + haf_n))$. For it to be of order 2, the truncation error $\tau_{n+1}(h)$ must satisfy $\tau_{n+1}(h) = O(h^2)$.

Recalling that $\tau_{n+1}(h) = \frac{X_{n+1} - U_{n+1}^*}{h}$, where U_{n+1}^* is the scheme starting from the exact solution $X(t_n) = X_n$, here, $U_{n+1}^* = X_n + h[b_1 f(t_n, X_n) + b_2 f(t_n + ah, X_n + ahf(t_n, X_n))]$. We will need the 2D Taylor formula: let G be a $\mathcal{C}^2$ function, then:

$$G(\alpha + h_1, \beta + h_2) = G(\alpha, \beta) + \underbrace{\frac{\partial G}{\partial \alpha}(\alpha, \beta)}_{\text{notation: } G_\alpha} h_1 + \underbrace{\frac{\partial G}{\partial \beta}(\alpha, \beta)}_{G_\beta} h_2 + O\left(\left(\sqrt{h_1^2 + h_2^2}\right)^2\right). \quad (2.35)$$

So, by the 2D Taylor formula,

$$\begin{aligned} & f(t_n + ah, X_n + ahf(t_n, X_n)) \\ &= f(t_n, X_n) + ah[f_t(t_n, X_n) + f_X(t_n, X_n)f(t_n, X_n)] + O(h^2). \end{aligned} \quad (2.36)$$

On the other hand, as $X'(t) = f(t, X(t))$, we also have

$$X''(t) = \frac{d}{dt}X'(t) = \frac{d}{dt}f(t, X(t)) = f_t(t, X(t)) + f_X(t, X(t))f(t, X(t)). \quad (2.37)$$

Therefore

$$\begin{aligned} & f(t_n + ah, X_n + ahf(t_n, X_n)) \\ &= f(t_n, X_n) + ahX''(t_n) + O(h^2) \\ &= X'(t_n) + ahX''(t_n) + O(h^2) \end{aligned} \quad (2.38)$$

The truncation error thus satisfies:

$$\begin{aligned}
 \tau_{n+1}(h) &= \frac{X_{n+1} - U_{n+1}^*}{h} \\
 &\stackrel{\text{Taylor-Lagrange}}{=} \frac{\overbrace{X(t_{n+1}) - X(t_n)} - hb_1X'(t_n) - hb_2(X'(t_n) + ahX''(t_n)) + O(h^3)}{h} \\
 &= \frac{hX'(t_n) + \frac{h^2}{2}X''(t_n) + O(h^3) - hb_1X'(t_n) - hb_2(X'(t_n) + ahX''(t_n)) + O(h^3)}{h} \quad (2.39) \\
 &= X'(t_n)(1 - b_1 - b_2) + h \left(\frac{X''(t_n)}{2} - ab_2X''(t_n) \right) + O(h^2) \\
 &= X'(t_n)(1 - b_1 - b_2) + hX''(t_n) \left(\frac{1}{2} - ab_2 \right) + O(h^2).
 \end{aligned}$$

For the method to be of order 2, it is necessary and sufficient that $b_1 + b_2 = 1$ and $ab_2 = 1/2$, which gives the conclusion. $\square$

Example 2.31 (Heun as a Second-Order RK Method). We apply the previous result for the case $b_1 = b_2$; then $b_1 = b_2 = \frac{1}{2}$ and $a = 1$:

$$\begin{array}{c|cc}
 0 & 0 & 0 \\
 1 & 1 & 0 \\
 \hline
 & \frac{1}{2} & \frac{1}{2}
 \end{array}$$

So: $K_1 = f(t_n, U_n) = f_n$ and $K_2 = f(t_n + h, U_n + hf(t_n, U_n)) = f(t_{n+1}, U_n + hf_n)$, thus $U_{n+1} = U_n + \frac{h}{2}[f_n + f(t_{n+1}, U_n + hf_n)]$, which is nothing else than the Heun method.

2.5.2 Consistency

The consistency of Runge-Kutta scheme can be resumed by a simple criterion given in the following proposition.

Proposition 2.32. Let f be a Lipschitz function. Then the Runge-Kutta method (explicit) is consistent if and only if $\sum_{i=1}^s b_i = 1$.

Proof. by Taylor expansions. $\square$

2.6 Multi-step Methods

The idea behind these schemes is to use previous steps (values) that are available.

Vocabulary: These schemes are also known as predictor-corrector methods.

Definition 2.33. The linear multi-step scheme of order s (with the notation $a_s = 1$) is given by the recurrence:

$$\sum_{k=0}^s a_k y_{n+k} = h \sum_{k=0}^s b_k f(t_{n+k}, y_{n+k}). \quad (2.40)$$

Here y_{n+s} is unknown, and the previous values $y_{n+s-1}, \dots, y_n$ are known.

We notice that if $b_s \neq 0$, then the method is implicit; otherwise, it is explicit. Let's consider some examples.

Example 2.34. Explicit Euler For $s = 1, a_0 = -1, a_1 = b_0 = 1, b_1 = 0$,

$$1 \cdot y_{n+1} + (-1) \cdot y_n = 1 \cdot hf(t_n, y_n) + 0 \cdot hf(t_{n+1}, y_{n+1}).$$

Example 2.35. Adams-Bashforth Two-Step (explicit) Taking $s = 2, a_0 = 0, a_1 = -1, a_2 = 1, b_0 = -\frac{1}{2}, b_1 = \frac{3}{2}, b_2 = 0$, the scheme is defined by the relation:

$$y_{n+2} = y_{n+1} + \frac{3}{2}hf(t_{n+1}, y_{n+1}) - \frac{1}{2}hf(t_n, y_n). \quad (2.41)$$

Example 2.36. BDF Two-Step (implicit) Here $s = 2, a_0 = \frac{1}{3}, a_1 = -\frac{4}{3}, a_2 = 1, b_0 = 0, b_1 = 0, b_2 = \frac{2}{3}$, and thus:

$$y_{n+2} - \frac{4}{3}y_{n+1} + \frac{1}{3}y_n = \frac{2}{3}hf(t_{n+2}, y_{n+2}). \quad (2.42)$$

Definition 2.37. The local truncation error is $\tau_{n+s}(h)$ defined by:

$$\tau_{n+s}(h) := \frac{\sum_{k=0}^s a_k X(t_{n+k}) - h \sum_{k=0}^s b_k f(t_{n+k}, X_{n+k})}{h}. \quad (2.43)$$

Reminder: The global truncation error is

$$\tau(h) = \max_n |\tau_{n+s}(h)|;$$

The multi-step scheme $(a_k, b_k)_{k=0}^s$ is consistent if $\lim_{h \rightarrow 0} \tau(h) = 0$.

Theorem 2.38. The multi-step scheme $(a_k, b_k)_{k=0}^s$ is consistent if and only if:

$$\sum_{k=0}^s a_k = 0, \quad \sum_{k=0}^s b_k = \sum_{k=0}^s k a_k. \quad (2.44)$$

or, equivalently:

$$\sum_{k=0}^{s-1} a_k = -1, \quad \sum_{k=0}^s b_k = s + \sum_{k=0}^{s-1} k a_k. \quad (2.45)$$

Proof. In (2.43) use Taylor expansions of X_{t+k} around the solution at the time t_{n+s} . $\square$

Example 2.39 (EE as a Consistent Multi-step Scheme). Taking the previous example where $a_0 = -1, a_1 = 1, b_0 = 1, b_1 = 0$, the conditions for the theorem are satisfied. Thus, the EE scheme is consistent.

Example 2.40 (Adam-Bashforth as a Consistent Multi-step Scheme). The Adam-Bashforth scheme satisfies

$$\sum_{k=0}^2 a_k = 0 \quad \text{and} \quad \sum_{k=0}^2 a_k k = \sum_{k=0}^2 b_k = 1. \quad (2.46)$$

Therefore, this scheme is consistent.

Example 2.41 (BDF is consistent). The BDF scheme ($s = 2$) satisfies $\sum_{k=0}^2 a_k = 0$ and

$$\sum_{k=0}^2 a_k k = \sum_{k=0}^2 b_k = 2/3. \text{ Hence, this scheme is also consistent.}$$

2.7 Application in Epidemiology: SIR Model

We quickly present the modeling leading to the SIR system. We refer to [4, 7, 5] and similar references for a more detailed presentation (just to change, we took a reference prior to the COVID-19 pandemic...). The variables are:

- S = the number of people susceptible to infection (the population not yet affected by the epidemic).

- I = the number of infected people.

- R = the number of people who have had the disease, died, or can no longer transmit it (having acquired immunity or being in quarantine, etc.).

Let $N = S(0) + I(0) + R(0)$ be the initial population (which will be conserved).

Model assumptions:

1. The number of infections (transition from S to I) is proportional to the number of individuals in S , the infection rate $I(t)/N$, and the duration Δt . We denote β as the proportionality factor. Since the number of new infections between t and $t + \Delta t$ is $S(t) - S(t + \Delta t)$, we obtain $S(t) - S(t + \Delta t) \simeq \beta SI \Delta t / N$.
2. The transition $I \rightarrow R$ is proportional to the number of individuals in I ; it depends on the recovery rate γ (here $1/\gamma$ is the average number of days before leaving the I compartment).

By taking the limit $\Delta t \rightarrow 0$, we obtain the system of equations, called the SIR model:

$$\frac{dS}{dt} = -\beta SI / N \quad (2.47)$$

$$\frac{dI}{dt} = \beta SI / N - \gamma I \quad (2.48)$$

$$\frac{dR}{dt} = \gamma I \quad (2.49)$$

We assume $S(0) = S_0 \neq 0$, $I(0) = I_0 > 0$, $R(0) = R_0 = 0$.

Remark 2.42. Since $\frac{d}{dt}(S + I + R) = 0$, then $S(t) + I(t) + R(t) = N = \text{cst.}$

A first question to ask is when I will be increasing or decreasing. Fortunately, this can be read directly from the equation for I : $I' = I(\beta S/N - \gamma)$, so it will increase when S/N is greater than $\frac{1}{\mathcal{R}_0}$, where $\mathcal{R}_0 = \beta/\gamma$ is known as the "reproduction rate." Note that $\mathcal{R}_0$ depends only on the characteristics of the disease and transmission and not on the state $S(t), I(t), R(t)$. In particular, if initially $S(0) \simeq N$, there will be no epidemic (in the sense that I will always be decreasing) if $\mathcal{R}_0 < 1$, and conversely, there will be an epidemic with exponential growth if $\mathcal{R}_0 > 1$.

Therefore, $\mathcal{R}_0$ is an important number, and in particular, it is the target of most epidemic containment policies; to make it sub-unitary, you can:

1. Make β small by reducing contacts (lockdown, etc.).
2. Make γ large by isolating the sick (so they are no longer contagious).
3. Otherwise, finally, make $S(0)$ smaller through vaccination if it is **effective and without side effects** (otherwise, it won't work).

In this (constant parameter!) model, the typical evolution of I , illustrated in Figure 1.2 on page 2, is as follows: it grows until a maximum value (corresponding to the "epidemic peak") and then decreases. Of course, in practice, the real behavior of epidemic data is more complicated because β will change depending on the preventive measures implemented, which in turn will fluctuate, etc.

The total number of infected individuals is

$$R_\infty := \lim_{t \rightarrow \infty} R(t) = I_0 + S_0 - \lim_{t \rightarrow \infty} S(t).$$

Note that $\lim_{t \rightarrow \infty} S(t)$ exists because $S(t), I(t), R(t) \geq 0, \forall t$ and S is decreasing).

Let ζ be the size of the epidemic. It can be shown (see [5]) that ζ is a solution of $1 - \frac{\zeta}{S(0)} = e^{-R_0(\zeta + I(0))}$.

Remark 2.43. Note that $S_\infty := \lim_{t \rightarrow \infty} S(t) \neq 0$; therefore, even in the absence of any protective measures, the epidemic will not affect everyone. This phenomenon is called the "herd immunity" of the SIR model. However, "not everyone" can still include too many people, and in practice, epidemic containment measures must be taken.

In practice: β and γ are unknown, so we proceed in 2 steps

- Inversion: find β, γ from observations $R(n), n = 1, \dots, N_{max}$
- Prediction: calculate $S(t), I(t), t \geq N_{max}$

Remark 2.44. Sometimes more complicated models are necessary, such as: $S \rightarrow E \rightarrow I \rightarrow R$ or as in [1].

2.8 Advanced considerations

2.8.1 Adaptive time steps for Runge-Kutta

The solution of some ODE can have very different regimes, sometimes it is almost constant, but other times it is highly variable. We would like to take advantage of the "calm" regions and use a large time step h , and a smaller one in highly oscillatory regions. For this, we need to employ **adaptive step** (i.e., variable and adjusted) methods. To determine how to choose this step, we need **error estimations**.

How to estimate the error in practice? The easiest way would be to double the step. One idea is to perform a calculation with a step $2h$ and compare it with two consecutive calculations with step h . Let Y_{2h} be the value obtained after a single step of size $2h$ and $Y_{h,h}$

after two steps of size h . Both are assumed to start from X_n or close to it within a tolerance. We know that, if the method is of order $p > 1$:

$$\begin{aligned} X(t_n + 2h) &= Y_{2h} + (2h)^{p+1}\psi_n + O(h^{p+2}) \\ X(t_n + 2h) &= Y_{h,h} + 2h^{p+1}\psi_n + O(h^{p+2}). \end{aligned}$$

The quantity $\Delta = Y_{2h} - Y_{h,h} = (2^{p+1} - 2)h^{p+1}\psi_n$ helps us adjust the h .

Remark 2.45. The availability of both Y_{2h} and $Y_{h,h}$ could provide an approximation of order $p + 1$ for $X(t_n + 2h)$. However, we do not want to use this new approximation but rather to estimate the error. So we refrain from improving the result Y_{2h} and only use Y_{2h} and $Y_{h,h}$ to estimate the error.

Although in principle the method above would be interesting, it uses too many evaluations of f . We will refine it by constructing two methods that use the same evaluations (thus same K_i) but whose linear combinations involving b_i yield different orders. We then talk about **embedded schemes of Runge-Kutta-Fehlberg (R-K-F)**. Notation:

$$\begin{array}{c|c} c & A \\ \hline & b^T \\ & \hat{b}^T \\ \hline & E \end{array}$$

where c, A, b give a scheme of order p while $c, A, \hat{b}$ give a scheme of order $p + 1$. The difference $E = b - \hat{b}$ serves to estimate the truncation error $\Delta = h \sum_{i=1}^s E_i K_i$.

The most popular are the R-K-F schemes of orders 4-5 or 5-6 or even 2-3. In practice, the algorithm is as follows:

- At the beginning, we specify a tolerance Δ_0 .
- If $\Delta > \Delta_0$ then we redo the calculation with the step $\tilde{h} = h \sqrt[p+1]{\frac{\Delta_0}{\Delta}}$.
- If $\Delta \leq \Delta_0$, the step h is kept the same or increased till $\Delta \sim \Delta_0$.

2.8.2 Systems of ODEs

Let $I \subset \mathbb{R}_+$ be an open interval and $F : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$. The Cauchy problem is written as :

$$\begin{cases} Y'(t) = F(t, Y(t)) \\ Y(t=0) = Y_0 \in \mathbb{R}^n \end{cases} \quad (2.50)$$

Example 2.46. $X'' = f(X)$ is not an ODE, but it can be written as a system, by setting: $Y_1 = X, Y_2 = X'$, and we get:

$$\begin{cases} Y_1' = Y_2 \\ Y_2' = f(Y_1) \end{cases}$$

Theorem 2.47 (existence and uniqueness). Let $F :]-\infty, \infty[\times \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a continuous function and Lipschitz with respect to the second variable

$$\|F(t, y) - F(t, \tilde{y})\| \leq L\|y - \tilde{y}\| \quad \forall t \in \mathbb{R}, \quad \forall y \in \mathbb{R}^n,$$

with an L not depending on $y \in \mathbb{R}^n$. Then the Cauchy problem (2.50) has a unique global solution (i.e., defined for all $t \geq 0$). If F is Lipschitz only around $(t_0 = 0, Y_0)$, then the solution is only locally defined.

Particular case: $F(t, y) = Ay$ with A being an $n \times n$ matrix. The problem

$$\begin{cases} Y' = AY \\ Y(0) = Y_0 \end{cases}$$

has the (unique) solution $Y(t) = e^{At}Y_0$ where we recall the definition of $e^{At} = \sum_{n=0}^{\infty} \frac{(At)^n}{n!}$. If A is diagonalizable, i.e.:

1) $\exists Q$ invertible such that $A = QDQ^{-1}$, D diagonal

2) or equivalently $\exists V_i, \lambda_i$, such that $AV_i = \lambda_i V_i$, $\|V_i\| = 1$, and $\{V_i; i = 1, \dots, n\}$ is a basis of $\mathbb{R}^n$

then, $Y(t) = \sum_{i=1}^n e^{\lambda_i t} V_i \langle y_0, V_i \rangle$.

2.8.3 System Stability

The stability turns out to be a very important question for systems. By setting $Z = Q^{-1}Y$, we obtain:

$$Y' = AY \implies Y' = QDQ^{-1}Y \implies Q^{-1}Y' = DQ^{-1}Y \quad (2.51)$$

And, since $Z' = (Q)^{-1}Y'$, we derive the following differential equation: $Z' = DZ$, and the problem:

$$\begin{cases} Z'_1 = \lambda_1 Z_1 \\ \vdots \\ Z'_n = \lambda_n Z_n \end{cases}$$

The solutions to this problem are given by: $Z_i(t) = e^{\lambda_i t} Z_i(0)$, and the stability of the system is equivalent to the stability of all the ODEs in the problem.

Example 2.48 (Explicit Euler). $U_{n+1} = U_n + hf(t_n, U_n)$. Let $f(y) = Dy$. Applying Explicit Euler to this example, we have: $U_{n+1} = U_n + hDU_n = (1 + hD)U_n$. Therefore, the scheme is stable if $|1 + h\lambda_i| < 1$ for all $i = 1, \dots, n$. In this case the stability condition is determined by the worse (read "largest") of the λ_i and having a large one will induce constraints on the h to be used for the method to remain stable.

Example 2.49 (Implicit Euler). Recall that the scheme is $U_{n+1} = U_n + hf(t_{n+1}, U_{n+1})$; for the previous example we have: $U_{n+1} = U_n + hDU_{n+1}$, so $U_{n+1} = (1 - hD)^{-1}U_n$. The scheme is stable if $|1 - h\lambda_i| > 1$ for all $i = 1, \dots, n$.

Advanced Topics 2.50 (Implementation). We always consider the case $f(t, y) = Ay$.

- For explicit schemes $U_{n+1} = U_n + hAU_n$, so it's a direct calculation.

- For implicit schemes $U_{n+1} = U_n + hAU_{n+1}$, $U_{n+1} = (I - hA)^{-1}U_n$; a linear system must be solved. For more complicated functions f , methods like Newton's or Picard's approximation are needed (cf. exercise 2.7) etc. ...

2.8.4 Stiff Systems

We have seen that implicit schemes are sometimes challenging to implement; why use them then? Consider the following differential system:

$$\begin{aligned} u' &= 998u + 1998v \text{ with } u(0) = 1 \\ v' &= -999u - 1999v \text{ with } v(0) = 0 \end{aligned}$$

We make the change of variables $u = 2y - z$ and $v = -y + z$, which gives us:

$$\begin{cases} y' = -y \\ z' = -1000z \end{cases} \Rightarrow \begin{cases} y(t) = e^{-t}y_0 \\ z(t) = e^{-1000t}z_0 \end{cases}$$

(so $\lambda_1 = -1$, $\lambda_2 = -1000$). Returning to our initial variables, we obtain the desired solution:

$$\begin{aligned} u &= 2e^{-t} - e^{-1000t}, \\ v &= e^{-t} + e^{-1000t}. \end{aligned}$$

For the stability of Explicit Euler, $|1 + h\lambda_1| < 1$ and $|1 + h\lambda_2| < 1$ are required, so $h \leq \frac{2}{1000}$.

For Implicit Euler stability, $|1 - h\lambda_i| > 1$, which is always satisfied ($\forall h > 0$). Assuming we are only interested in the e^{-t} part of the solution (treating e^{-1000t} as a perturbation, which it is), the precision of both schemes could be good for sufficiently large steps h ; however, for Explicit Euler, we must use a small h as stability is not guaranteed otherwise. Conclusion: using implicit schemes allows solving with a larger h , hence more quickly.